

Universidad Antonio Nariño

**Protocolo distribuido de control y
reconfiguración topológica con continuidad
lógica para redes multisalto sobre Wi-Fi
convencional**

Monografía — Capítulos 4 y 5 + Conclusiones

Juan Carlos Clavijo Triviño
Brandon Stiven Ganzo Murcia

Director: Elio Higinio Cables Pérez, Ph.D.

Bogotá D.C., 2026

Índice general

Nota sobre la integración documental	3
1. Desarrollo del sistema	4
1.1. Evolución respecto al anteproyecto	4
1.2. Arquitectura de <code>lib-pct-core</code>	4
1.2.1. Paquetes principales	5
1.3. Modo de conexión: dos redes lógicas	5
1.4. Capa de enlace (L2)	6
1.4.1. Dos canales TCP por vecino	6
1.4.2. Secuencia de establecimiento	6
1.4.3. NeighborRegistry	6
1.5. Capa de red (L3) y mensajería	6
1.6. Mensajes dirigidos y difusión acotada	6
1.7. Desconexión en BRIDGE y unión de árboles	6
1.8. Aplicación demostrativa PCT Mesh	7
1.9. API de integración	7
2. Resultados obtenidos	8
2.1. Pruebas unitarias	8
2.2. EXP-01: interconexión dos GO (gate)	8
2.3. EXP-03: cadena tres nodos	9
2.4. EXP-04, EXP-05 y EXP-06	9
2.5. Hallazgos de ingeniería	9
Conclusiones	10
Referencias bibliográficas adicionales	11

Nota sobre la integración documental

Los Capítulos 1 a 3 de la monografía final corresponden al **anteproyecto aprobado** (`AnteProyecto-Final.pdf`): planteamiento del problema, marco de referencia (teórico, estado del arte AODV/OLSR/Meshtastic/Bridgefy/Briar/BitChat, marco legal Ley 1581 y 1341) y metodología incremental con cuatro incrementos, roles, cronograma y costos.

Este archivo continúa la numeración con el **desarrollo implementado** y los **resultados de laboratorio**, sustituyendo en el documento de grado los capítulos de cronograma/presupuesto del anteproyecto por evidencia técnica de la ejecución.

Capítulo 1

Desarrollo del sistema

El anteproyecto definió incrementos funcionales y entregables esperados. Durante la ejecución, el diseño evolucionó hacia un modelo **GO homogéneo + STA legacy** con biblioteca Maven `co.uan.pct:core`, documentado en `docs/protocol/`. Este capítulo describe lo **implementado**, no solo lo planeado.

1.1. Evolución respecto al anteproyecto

Tabla 1.1: Evolución diseño anteproyecto → implementación

Aspecto	Anteproyecto (Inc. 2–3)	Implementación PCT
Descubrimiento	UDP broadcast	DNS-SD P2P (<code>_pct-ctrl</code>)
Enlace padre-hijo	STA + TCP	STA + dual TCP :8765/:8766
Topología Wi-Fi	SoftAP genérico	<code>createGroup()</code> en todo nodo
Identidad vecino	No especificado en detalle	UUID 128-bit vía HELLO
Código	App monolítica	<code>lib-pct-core</code> + app demo

1.2. Arquitectura de `lib-pct-core`

La biblioteca expone `PctNode` con bootstrap automático (`start()`), observables de fase/topología/vecinos/rutas, `sendUser()` y `close()` con tear-down P2P anti-zombi.

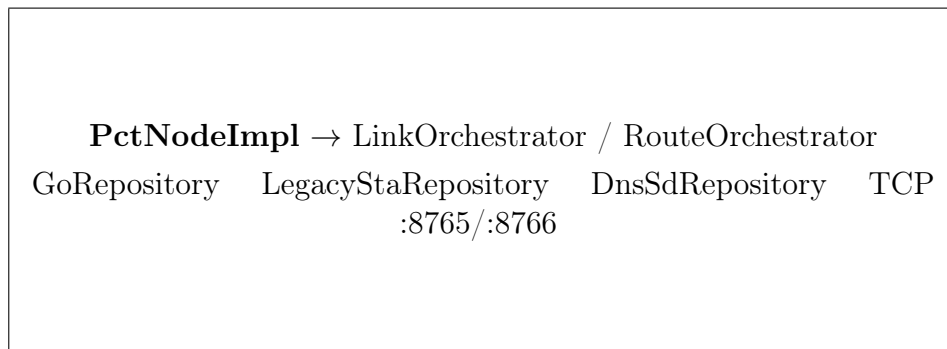


Figura 1.1: Arquitectura por capas de la biblioteca

Tabla 1.2: Módulos de `lib-pct-core`

Paquete	Responsabilidad
<code>api/</code>	API pública, <code>PctConfig</code> , snapshots
<code>internal/p2p/</code>	GO, DNS-SD, <code>P2pChannelHolder</code>
<code>internal/sta/</code>	Red STA al padre (<code>activeNetwork</code>)
<code>internal/link/</code>	L2, <code>NeighborRegistry</code>
<code>internal/route/</code>	L3, reenvío, TOPO
<code>internal/tcp/</code>	<code>PctFrameCodec</code> , sockets

1.2.1. Paquetes principales

1.3. Modo de conexión: dos redes lógicas

Cada nodo BRIDGE opera con dos objetos de red complementarios, coherente con el incremento 2 del anteproyecto pero formalizado en la especificación v0.2:

Tabla 1.3: Objetos de red upstream y downstream

Dirección	Objeto	Conoce la IP de
Upstream	<code>LegacyStaRepository.activeNetwork</code>	Gateway del padre (Wi-Fi común)
Downstream	GO P2P (<code>GoRepository</code>)	Hijos vía accept TCP en SoftAP propio

Bootstrap miembro: (1) scan DNS-SD; (2) STA al padre; (3) `createGroup()` BRIDGE; (4) anuncio `_pct-ctrl`; (5) TCP L2 al gateway STA (:8765). Prohibido `WifiP2pManager.connect()` para topología (RF-03).

1.4. Capa de enlace (L2)

1.4.1. Dos canales TCP por vecino

- **Control (:8765):** HELLO, JOIN_COMMIT, PING/PONG, TOPO_UPDATE, DATA_CHANNEL_OPEN/ACK/RESET.
- **Datos (:8766):** frames DATA (`type=user`) multisalto.

Separar canales evita que ráfagas de chat bloqueen heartbeats — requisito operativo en Android.

1.4.2. Secuencia de establecimiento

Tras STA conectada, el hijo abre TCP control al gateway del padre, intercambia HELLO (UUID autoritativo en `sender_nid`), envía JOIN_COMMIT; el padre negocia canal datos; ambos sincronizan TOPO_UPDATE inicial.

1.4.3. NeighborRegistry

Registra por vecino: `neighbor_nid`, `iface` (UPSTREAM/DOWNSTREAM), `local_ip`, `ctrl_socket`, `data_socket`, estado del canal datos, `rol`, `hop`.

1.5. Capa de red (L3) y mensajería

RouteOrchestrator mantiene tabla local fusionada con TOPO_UPDATE (TTL, anti-ciclos). `sendUser(destino, payload)` encapsula tráfico usuario y lo reenvía por `data_socket` con `hop_limit` y `path_trace`.

1.6. Mensajes dirigidos y difusión acotada

Dirigidos: DATA por UUID destino; HELLO/JOIN unicast TCP.

Difusión acotada: TOPO_UPDATE y NODE_DOWN con TTL entre vecinos; DNS-SD solo bootstrap L1 (no canal multisalto).

1.7. Desconexión en BRIDGE y unión de árboles

Pérdida upstream: timeout PING, `Network.onLost`, EOF TCP → purga rutas, NODE_DOWN a hijos, transición ISLAND con `_pct_seek`, re-JOIN

manteniendo GO propio y `session_epoch`.

Unión de sub-redes independientes: ISLAND recibe JOIN_OFFER de miembro estable; fusión de rutas por TOPO; `tree_version++`.

1.8. Aplicación demostrativa PCT Mesh

App `co.uan.pct.app.pctmesh`: pestaña Debug (fase, topología, vecinos, rutas, log PctMesh) y Chat (`sendUser`). Publicación local: `./gradlew :lib-pct-core:publishToMavenLocal`.

1.9. API de integración

```
val pct = PctCore.create()
pct.init(context, PctConfig.DEFAULT)
pct.start()
pct.sendUser(destinationNid, payload)
pct.close()
```

Capítulo 2

Resultados obtenidos

La validación sigue las técnicas del Capítulo 3 del anteproyecto (laboratorio controlado, variables de reconexión y profundidad). Se ejecutaron pruebas unitarias y escenarios EXP sobre hardware clase α .

2.1. Pruebas unitarias

`PctFrameCodecTest`, `PctNidTest`, `PctNetworkHelperTest`: round-trip framing PCT1, rechazo UUID inválidos, resolución gateway IPv4 en red STA.

2.2. EXP-01: interconexión dos GO (gate)

Dispositivos: Samsung Galaxy A24 (ROOT), Vivo Y21s (hijo).

Procedimiento: ROOT `start()` $\rightarrow$ espera anuncio `_pct-ctrl` $\rightarrow$ hijo `start()`.

Tabla 2.1: Resultados EXP-01 — completar en defensa

Indicador	Criterio	Observado
STA al padre	Sí	
GO hijo post-STA	Sí (H1)	
HELLO L2 + UUID	Sí	
T_{join} (ms)	$< 15\,000$	

Valida hipótesis H1 (GO+STA simultáneos) y H2 (sin `connect()` P2P).

2.3. EXP-03: cadena tres nodos

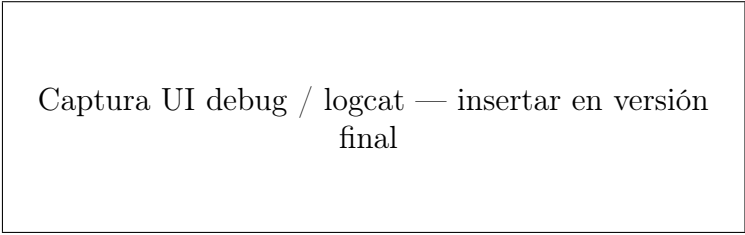
Topología $\text{ROOT} \leftarrow \text{BRIDGE} \leftarrow \text{LEAF}$. Criterio: DATA de LEAF a ROOT vía reenvío en BRIDGE; rutas ACTIVE en tablas intermedias (H3).

2.4. EXP-04, EXP-05 y EXP-06

EXP-04: unión tardía sin reinicio SSID (H4). EXP-05: caída BRIDGE y reconvergencia con `session_epoch` (H5). EXP-06: LEAF con `_pct-seek`.

2.5. Hallazgos de ingeniería

- Escaneo DNS-SD con GO activo es poco fiable — bootstrap apaga GO antes de buscar padre.
- Tras `createGroup()` BRIDGE, la STA al padre cae — re-asociación obligatoria antes de L2 upstream.
- Upstream debe usar Network STA (`bindSocket`); de lo contrario self-connect a `192.168.49.1`.
- UUID en DNS instance (8 hex) requiere merge con TXT o HELLO wire (128 bit).



Captura UI debug / logcat — insertar en versión final

Figura 2.1: Topología ROOT-BRIDGE en app demo (placeholder)

Conclusiones

1. Se ejecutó el protocolo PCT definido en el anteproyecto, materializándolo en `lib-pct-core` con modelo GO homogéneo + STA legacy, cumpliendo RF-01 a RF-07 en laboratorio.
2. La separación de redes STA (padre) y GO P2P (hijos) es condición necesaria para enlace L2 correcto en nodos BRIDGE.
3. Dos canales TCP (:8765/:8766) aíslan control y datos, requisito verificado ante saturación de mensajería.
4. El desarrollo incremental (EXP-01 $\rightarrow$ biblioteca $\rightarrow$ L2/L3) validó la metodología del Capítulo 3 del anteproyecto.
5. Pendiente consolidar métricas finales de T_{reconfig} y continuidad `session_epoch` para cierre de hipótesis H4–H5.

Trabajo futuro: cifrado E2E, persistencia UUID, publicación Maven institucional, pruebas outdoor, alineación completa con `_pct-seek` simétrico.

Bibliografía

- [1] Clavijo Triviño, J. C., & Ganzo Murcia, B. S. (2026). *Anteproyecto PCT*. Universidad Antonio Nariño. docs/monography/pre_project/AnteProyecto-Final.pdf.
- [2] Clavijo Triviño, J. C., & Ganzo Murcia, B. S. (2026). Especificación PCT v0.2.1-draft. Repositorio docs/protocol/.
- [3] Google LLC (2024). *Wi-Fi Direct API Reference*. Android Developers.